DESARROLLO DE APLICACIONES II

Patrones y documentación

Grupo 9 · Core

20 de octubre de 2026

Contenido

Parte A · Patrones en el dominio 3
1 · La entidad central y su máquina de estados 3
2 · Los patrones, con su ADR 4
3 · Auditoría SOLID: una violación real y su refactor 5
Parte B · Esqueleto documental 6
4 · BRD — índice y tres requisitos 6
5 · Tres historias de usuario con criterios Gherkin 7
6 · Manual — índice por rol, clasificado según Diátaxis 9
7 · Matriz de trazabilidad 10
Bonus · El State implementado con su test de 409 10
Cómo reproducirlo 11
Supuestos 11

Desarrollo de Aplicaciones II · Clase 12 · Actividad integradora

Dominio: **Core**, el pasamanos de eventos de la plataforma municipal. Recibe los eventos asincrónicos de los 9 módulos, valida el sobre, guarda evidencia y los entrega a quien esté suscripto.

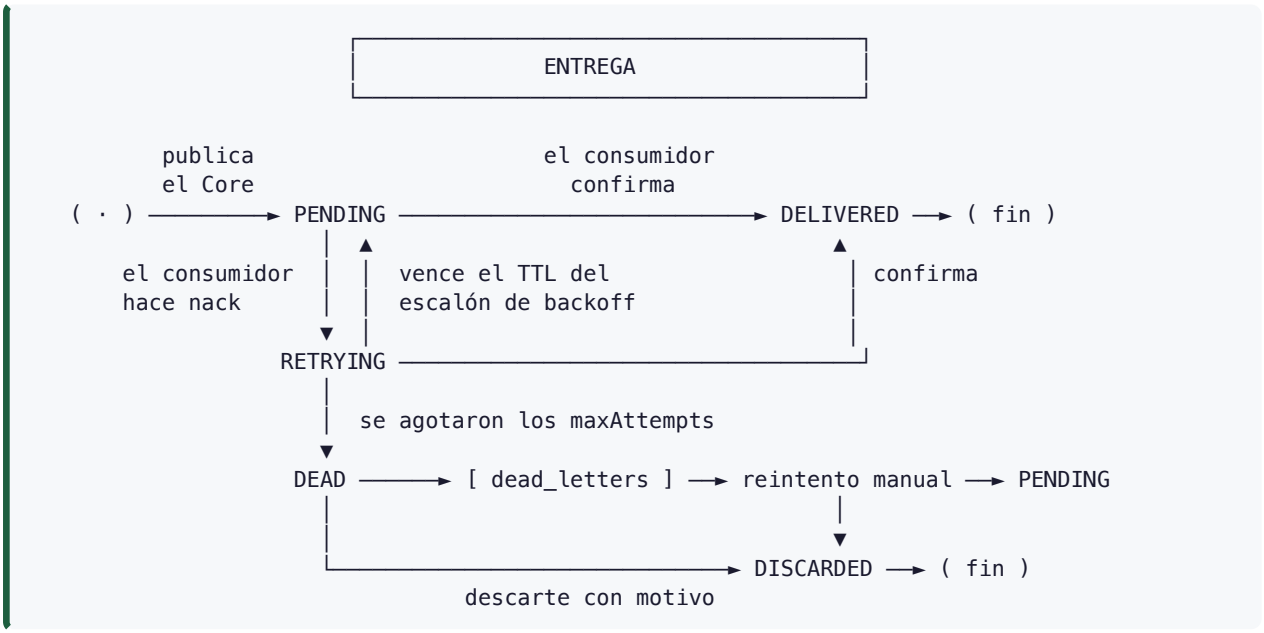
> **Lo que hay acá no es una propuesta: está implementado y probado.** El refactor SOLID de la Parte A se hizo, los cuatro patrones están fijados con pruebas que fallan si alguien deshace la decisión, y el Bonus —transición inválida con 409— tiene su test. 186 pruebas, 91% de cobertura.

Parte A · Patrones en el dominio

1 · La entidad central y su máquina de estados

La entidad central del Core **no es el evento: es la entrega**.

Un evento que llega se copia en una entrega por cada módulo suscripto, y cada una tiene su propia vida. El mismo `ticketCreated` puede estar **entregado** para Obras y **muerto** para Rentas, con tres reintentos en el medio. Modelar el evento como la entidad con estado haría imposible representar eso.



Y el ciclo de la **dead letter**, que es lo que ve el equipo en el dashboard:

```
OPEN ──► RETRIED      (reintento manual exitoso)
  └────► DISCARDED    (descarte con motivo obligatorio)

RETRIED y DISCARDED son terminales: cualquier transición desde ahí es 409.
```

La interfaz de estados

Estado	Qué significa	Transiciones permitidas
`PENDING`	Publicada al broker, sin confirmar	→ `DELIVERED`, `RETRYING`, `DISCARDED`
`RETRYING`	Rechazada, esperando el próximo escalón	→ `DELIVERED`, `RETRYING`, `DEAD`
`DELIVERED`	El consumidor confirmó	terminal
`DEAD`	Agotados los intentos; está en la DLQ	→ `PENDING` (reintento manual), `DISCARDED`
`DISCARDED`	Cerrada a mano con motivo	terminal

Dos reglas que la tabla hace explícitas y que antes estaban implícitas:

- **De `DELIVERED` no se sale.** Reintentar algo ya entregado duplicaría el efecto en el módulo destino, que es justamente lo que la regla 1 del enunciado prohíbe.
- **`DEAD` no es el final.** Es el estado donde la decisión pasa a una persona.

---

2 · Los patrones, con su ADR

Cuatro decisiones, cada una con su ADR en `/docs/adr` y su prueba.

Patrón	Tipo	Dónde vive	ADR
Pure Fabrication + Indirection	GRASP	`app/repositories/`	ADR-004
Strategy	GoF · comportamiento	`app/services/retry_policy.py`	ADR-002
State	GoF · comportamiento	`app/models/events.py` + `delivery_service.py`	ADR-001
Adapter	GoF · estructural / integración	`app/messaging/`	ADR-003

GRASP · Fabricación pura — la capa de repositorios

Por *Experto en Información*, un `EventLog` sabría buscarse y paginarse solo. Llevado al código, eso termina en entidades que saben de sesiones y de dialectos de SQL. Acá se nota más que en otros módulos: el Core corre sobre **SQLite en las pruebas y PostgreSQL desplegado**, y hay consultas que difieren entre los dos.

Se fabrica una capa que no corresponde a ningún concepto del dominio. No existe un «repositorio» en el municipio; existe porque concentra el acceso a datos.

Qué compró, concretamente: las dos diferencias reales entre motores quedaron encapsuladas — `JSONType` con su variante `JSONB`, y `hourly_volume`, que agrupa en Python en vez de usar la función de truncado de cada motor— y ningún servicio se enteró.

Strategy · la política de reintentos

Es el patrón que salió del refactor SOLID del punto 3. Ver ahí.

State · el ciclo de vida de la entrega

Lo del punto 1. Una transición que la máquina no permite es un **conflicto con el estado actual**, no un error del cliente ni una falla del servidor: 409.

Adapter · el broker

El Core es un pasamanos: la mensajería no es un detalle, es el trabajo. Pero si el vocabulario de `aio-pika` entra en los servicios, la lógica del hub queda atada a la librería y **sin broker levantado no se podría probar nada**.

Un contrato propio, `Broker`, con siete operaciones en el vocabulario del Core, y dos implementaciones: `RabbitMQBroker` adapta `aio-pika`; `FakeBroker` es un broker en memoria que rutea de verdad, con exchanges, colas, bindings y comodines de topic.

Qué compró: la suite corre completa sin RabbitMQ y el pipeline de ingesta se ejercita en cada commit.

Qué cuesta, y lo decimos: el contrato es el mínimo común. Cosas específicas de RabbitMQ no se pueden usar sin agregarlas antes a la interfaz. Y hay un riesgo que el patrón no cubre solo — que `FakeBroker` diverja y dé falsa confianza —, por eso el camino AMQP real se verifica aparte antes de cada entrega.

---

3 · Auditoría SOLID: una violación real y su refactor

La violación

Principio abierto/cerrado (OCP). La regla de cuánto esperar antes del próximo intento estaba escrita **dos veces, idéntica**, en dos servicios distintos:

```
app/services/event_hub_service.py
app/services/delivery_service.py
```

```python
def _delay_for(attempt: int) -> int:
    delays = settings.retry_delays or [5]
    return delays[min(max(attempt, 1), len(delays)) - 1]
```

No es sólo repetición. Significa que **cambiar la política obliga a modificar los dos servicios**, y que nada garantiza que sigan de acuerdo. Para extender el comportamiento hay que editar código existente y probado.

Y tenía fecha de vencimiento: el día que hiciera falta backoff exponencial, o *jitter* para que los nueve módulos no reintenten todos en el mismo instante, habría que escribirlo en los dos lados otra vez.

Antes / después

	Antes	Después
Dónde vive la regla	duplicada en 2 servicios	`app/services/retry_policy.py`
Agregar una política	editar los 2 servicios	escribir una clase, no tocar nada
Probar con otra política	cambiar config global	inyectarla por constructor
Riesgo de divergencia	real, nada lo impide	imposible: hay una sola copia

Después:

```python
class RetryPolicy(ABC):
    @abstractmethod
    def delay_for(self, attempt: int) -> int: ...

    def exhausted(self, attempt: int, max_attempts: int) -> bool:
        return attempt >= max_attempts


class TieredBackoff(RetryPolicy):
```

```
    """Escalones fijos de RETRY_DELAYS. Si se piden más intentos que
    escalones, repite el último: `maxAttempts` lo elige cada módulo y no
    tiene por qué conocer nuestra lista."""
```

Los dos servicios la reciben por constructor:

```
self.retry_policy = retry_policy or default_retry_policy()
```

Qué otros principios toca

- **SRP.** Los servicios dejaron de ser responsables de *calcular* el backoff; sólo lo aplican.
- **DIP.** Dependen de la abstracción `RetryPolicy`, no de una función concreta.

La regresión está fijada

```
def test_la_politica_quedo_en_un_solo_lugar():
    culpables = [f.name for f in servicios.glob("*.py")
                 if f.name != "retry_policy.py" and "def _delay_for" in f.read_text()]
    assert culpables == []
```

No es un comentario pidiendo que no se repita: si vuelve a aparecer, la suite falla.

Parte B · Esqueleto documental

4 · BRD — índice y tres requisitos

Índice

```
1.  Resumen ejecutivo
2.  Contexto y problema
    2.1  La plataforma municipal y sus 9 módulos
    2.2  Qué pasa hoy sin un hub de eventos
3.  Alcance
    3.1  Dentro: ingesta, validación de sobre, ruteo, evidencia, DLQ,
         catálogos globales
    3.2  Fuera: reglas de negocio de las áreas, identidad de ciudadanos
4.  Interesados
5.  Requisitos de negocio (BR)
6.  Restricciones y supuestos
7.  Riesgos
8.  Criterios de aceptación
9.  Glosario
10. Anexos: contrato del sobre, catálogo de eventos
```

Tres requisitos de negocio

ID	Requisito	Métrica	MoSCoW
BR-01	Ningún evento aceptado se pierde, aunque el módulo destino esté caído	100% de los eventos aceptados terminan en `DELIVERED` o quedan visibles en la DLQ. Cero eventos sin rastro.	**Must**
BR-02	Un evento procesado no genera efectos duplicados	0 entregas repetidas del mismo `eventId` al mismo módulo, medido sobre el log de entregas	**Must**
BR-03	Cada equipo puede diagnosticar su integración sin pedirle nada al equipo 9	El tablero muestra estado por evento y las alertas de integración; tiempo de diagnóstico < 5 min sin intervención nuestra	**Should**

5 · Tres historias de usuario con criterios Gherkin

HU-01 · Que un módulo caído no me haga perder eventos

> **Como** backend de un módulo que publica **quiero** que el Core acepte mi evento aunque el destinatario esté caído **para** no tener que implementar yo la lógica de reintentos.

*Traza a BR-01. Patrón: **State** + **Strategy**.*

```
Escenario: el consumidor rechaza y el evento se reintenta
  Dado un evento aceptado por el Core y entregado a "obras"
  Cuando el consumidor de "obras" lo rechaza con nack
  Entonces la entrega queda en estado RETRYING
  Y se programa el próximo intento según el escalón que corresponda
  Y el evento sigue teniendo evidencia en la bitácora

Escenario: se agotan los intentos y la entrega queda visible
  Dado un evento con 4 intentos fallidos de entrega a "obras"
  Cuando vence el último intento
  Entonces la entrega queda en estado DEAD
  Y aparece en GET /api/v1/dlq con su motivo
  Y nunca se pierde: el cuerpo original queda guardado
```

HU-02 · Que reenviar un evento no duplique el efecto

> **Como** backend de un módulo que publica **quiero** que reenviar un evento ya procesado no haga nada **para** poder reintentar sin miedo cuando no sé si llegó.

*Traza a BR-02. Patrón: **State**.*

```
Escenario: el mismo eventId llega dos veces
  Dado un evento con eventId "3f6c1b7e-..." ya procesado
  Cuando se publica otra vez el mismo eventId
  Entonces el Core responde 200 y no 202
  Y lo marca como DUPLICATE
  Y no genera ninguna entrega nueva
```

```
Escenario: reintentar una dead letter ya resuelta
  Dada una dead letter en estado DISCARDED
  Cuando un administrador intenta reintentarla
  Entonces el Core responde 409 Conflict
  Y el estado de la dead letter no cambia
```

HU-03 · Que yo mismo pueda ver por qué no me llega un evento

> **Como** integrante de un módulo **quiero** ver el estado de cada evento mío y los huecos de integración **para** diagnosticar sin depender del equipo 9.

*Traza a BR-03. Patrón: **Pure Fabrication** (el tablero se arma sobre las consultas de los repositorios, sin lógica de negocio nueva).*

```
Escenario: ver el estado de una entrega
  Dado que pertenezco al módulo "obras"
  Cuando consulto GET /api/v1/events/{eventId} de un evento mío
  Entonces veo el estado de cada entrega, módulo por módulo
  Y veo el motivo si alguna fue rechazada

Escenario: descubrir que espero un evento que nadie publica
  Dado que "obras" está suscripto a "streetClosureEnded"
  Y ningún módulo declara publicar ese tipo
  Cuando consulto GET /api/v1/dashboard/integration-alerts
  Entonces aparece una alerta NO_CONSUMER_DECLARED para ese tipo

Escenario: no puedo ver tráfico ajeno
  Dado que pertenezco al módulo "obras"
  Cuando consulto un evento en el que no soy origen ni destinatario
  Entonces el Core responde 404
```

---

6 · Manual — índice por rol, clasificado según Diátaxis

Sección	Rol	Diátaxis	Por qué ahí
Publicar tu primer evento en 10 minutos	Dev de módulo	**Tutorial**	Se aprende haciendo, con un resultado garantizado
Suscribirse a un tipo de evento	Dev de módulo	**How-to**	Tarea concreta de alguien que ya sabe qué quiere
Reintentar o descartar una dead letter	Admin (equipo 9)	**How-to**	Ídem, con permisos de admin
Conectarse al Core desplegado	Dev de módulo	**How-to**	Pasos para un objetivo puntual
Contrato del sobre de eventos	Dev de módulo	**Referencia**	Se consulta, no se lee de corrido
Catálogo de los 73 tipos de evento	Dev de módulo	**Referencia**	Ídem
Endpoints de la API (Swagger)	Dev de módulo / Admin	**Referencia**	Generada del código
Barrios, zonas y catálogos globales	Dev de módulo	**Referencia**	Datos que se consultan
Por qué el backoff no usa scheduler	Admin / Arquitecto	**Explicación**	Entender una decisión, sin tarea asociada
Por qué un tipo no declarado no se rechaza	Arquitecto	**Explicación**	Ídem
Los ADR	Arquitecto	**Explicación**	El registro de las decisiones
Alertas de integración: qué significa cada una	Dev de módulo / Admin	**Explicación**	Interpretar, no ejecutar

La clasificación no es decorativa: separa **lo que se lee una vez** (tutorial) de **lo que se consulta siempre** (referencia), que es el error más común en la documentación de una API y el que hace que nadie la lea.

7 · Matriz de trazabilidad

BR	HU	Criterio de aceptación	Patrón	Componente	Prueba
BR-01	HU-01	El consumidor rechaza → `RETRYING`	State	`delivery_servi`	`test_dlq_reintentos.py`
BR-01	HU-01	Próximo intento según el escalón	Strategy	`retry_policy.p`	`test_patrones.py::test_la`
BR-01	HU-01	Agotados los intentos → `DEAD` y visible en DLQ	State	`delivery_servi`	`test_dlq_reintentos.py`
BR-01	HU-01	El evento sobrevive al broker caído	Adapter	`messaging/broker.py`	`test_hub.py`
BR-02	HU-02	Mismo `eventId` dos veces → `DUPLICATE`, 200	State	`event_hub_serv`	`test_hub.py`
BR-02	HU-02	Transición inválida → **409 Conflict**	State	`delivery_servi`	`test_patrones.py::test_ur`
BR-03	HU-03	Estado por entrega, módulo por módulo	Pure Fabrication	`repositories/event_repository.py`	`test_scope_y_dashboard.py`
BR-03	HU-03	Alerta `NO_CONSUMER_DECLARED`	Pure Fabrication	`stats_service.`	`test_similar_names.py`
BR-03	HU-03	No se ve tráfico ajeno → 404	Pure Fabrication	`event_reposito`	`test_scope_y_dashboard.py`

Bonus · El State implementado con su test de 409

```python
async def test_una_transicion_invalida_responde_409(client, auth, make_envelope, broker):
    admin = await auth("core")

    # Provocar una dead letter real: un evento cuyo `data` no cumple el schema.
    await client.put("/api/v1/event-types/ticketCreated/schema",
                     json={"jsonSchema": {"type": "object", "required": ["ticketId"]}},
                     headers=admin)
    await client.post("/api/v1/events",
                      json=make_envelope(eventType="ticketCreated", data={}),
                      headers=await auth("atencion-ciudadana"))

    dead_letter_id = (await client.get("/api/v1/dlq?status=OPEN",
                                       headers=admin)).json()["items"][0]["id"]
```

```
    primero = await client.post(f"/api/v1/dlq/{dead_letter_id}/discard",
                                json={"reason": "duplicado de otro reclamo"},
headers=admin)
    assert primero.status_code == 200

    segundo = await client.post(f"/api/v1/dlq/{dead_letter_id}/retry", headers=admin)
    assert segundo.status_code == 409
    assert segundo.json()["code"] == "CONFLICT"
```

No simula la dead letter: la **provoca**, cargando un schema y publicando un evento que no lo cumple. Después la descarta y verifica que el segundo intento choca con el estado.

Cómo reproducirlo

```
cd backend
.venv/bin/python -m pytest tests/test_patrones.py -v
.venv/bin/python -m pytest -q --cov
```

Supuestos

- **Grupo 9 / dominio Core.** El Core es el módulo 9 y figura como «Equipo 9» en el catálogo, así que el archivo se nombró `Clase12_Grupo9_Core_Patrones.pdf`. Si la cátedra numera distinto, se cambia el nombre y nada más.
- **El State ya existía parcialmente** como guardas `if` dentro de los métodos que cambian el estado. Lo que se hizo acá fue explicitarlo como tabla de transiciones y documentarlo; el 409 ya se devolvía y ahora tiene test propio.